App Development Project Report

App Dev Project Report

1. Student Details

Name: Mohit
Roll Number: BS24F2001273
Email: 24f2001273@ds.study.iitm.ac.in

About Me: I am a student in the IIT Madras BS Degree program with a strong interest in full-stack web development and scalable architectures. I enjoy building applications that solve real-world logistical problems by combining robust backend logic with clean, responsive user interfaces.

---

2. Project Details

Project Title: Quantified Self App

Problem Statement: BarejaHospitals

To design and build a comprehensive, role-based web application that streamlines hospital operations. The system allows patients to search for doctors and book appointments, enables doctors to manage their availability and log patient treatments, and provides administrators with oversight tools to manage staff and users.

Approach:

The application utilizes a decoupled architecture. The backend is built using Flask as a RESTful API, implementing JWT for secure role-based access control (RBAC). It integrates Celery and Redis to handle asynchronous background tasks (like PDF generation). The frontend is built as a Single Page Application (SPA) using Vue.js and Bootstrap 5 to deliver a seamless, responsive user experience.

---

3. AI/LLM Declaration

I used Gemini to assist me in architectural planning, debugging Flask logic errors (such as resolving database integrity constraints), generating some modal codes for Vue.js components, and some styling. The extent of AI/LLM usage is around **15–20%**. All core business logic specifically, the double-booking prevention algorithms, Celery background job integration, and state management was thoroughly reviewed, modified, and manually integrated by me to fit the specific requirements of this project's rubric.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Vue.js 3	Core frontend framework for building the Single Page Application (SPA).
Flask	Core backend web framework for building RESTful APIs.
Flask-SQLAlchemy	Object Relational Mapper (ORM) for managing the SQLite database.
Flask-JWT-Extended	Managing secure user authentication and role-based access tokens.
Celery	Asynchronous task queue for handling scheduled background jobs.
Redis	In-memory data structure store used as a message broker for Celery and for API caching.t
Bootstrap 5	Frontend styling, modal generation, and mobile-responsive design.
xhtml2pdf	Backend library used within Celery tasks to generate downloadable PDF reports.

---

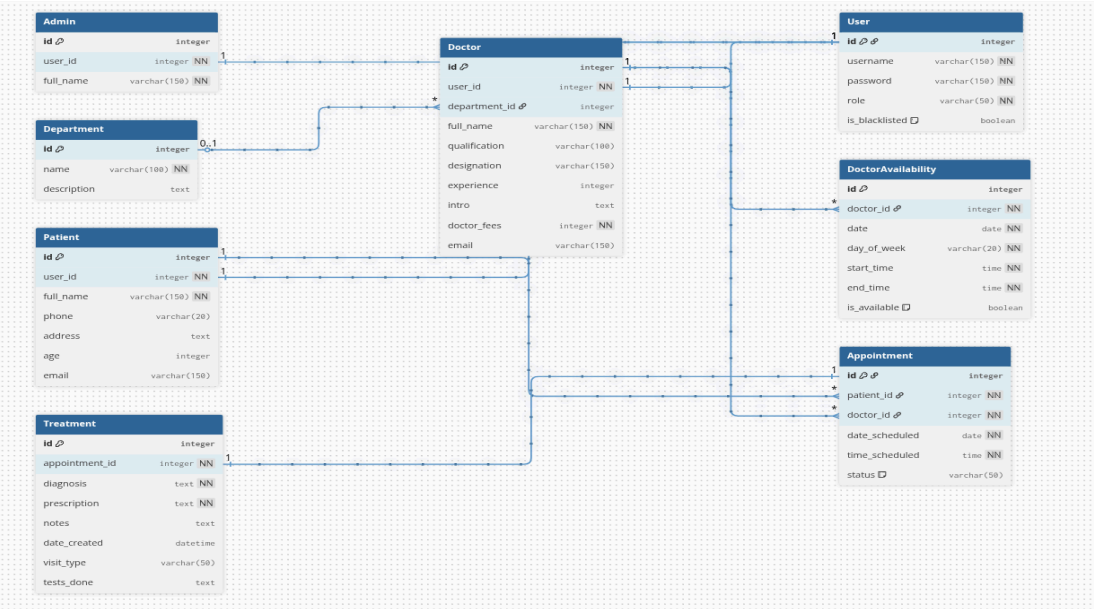
5. Database Schema / ER Diagram

Tables:

1. **User:** Stores system credentials and roles (id, username, password_hash, role, is_blacklisted).
2. **Doctor:** Stores doctor profiles (id, user_id, department_id, full_name, fees, experience).
3. **Patient:** Stores patient profiles (id, user_id, full_name, phone, age).
4. **Department:** Stores hospital specializations (id, name).
5. **Appointment:** Logs booked slots (id, patient_id, doctor_id, date, time, status).
6. **Treatment:** Logs medical notes (id, appointment_id, diagnosis, prescription).

Relationships:

- One-to-One → User to Doctor / User to Patient
- One-to-Many → Department to Doctor
- One-to-Many → Doctor to Appointment / Patient to Appointment
- One-to-One → Appointment to Treatment



---

6. API Resource Endpoints

Endpoint	Method	Description
`/api/login`	POST	Authenticates user and generates a JWT access token.
`/api/doctor/treatment/<id>`	POST	Marks an appointment as completed and saves medical notes.
`/api/admin/dashboard`	GET	Fetches aggregated system statistics and user lists (Cached via Redis).
`/api/patient/export_history`	POST	Triggers an asynchronous Celery task to generate a CSV export.

YAML API Definition File:
Included separately in the submission ZIP as `api.yaml`.

---

7. Architecture and Features (optional)

Architecture Overview:

- **`/backend/app.py`** – Main Flask application, API routing, and Celery configuration.
- **`/backend/models.py`** – Database schema definitions using SQLAlchemy.
- **`/frontend/src/views/`** – Vue.js dashboard components separated by user role.
- **`/frontend/src/router/`** – Vue Router handling protected frontend navigation.

Implemented Features:

- JWT-based Authentication with strict Role-Based Access Control (Admin, Doctor, Patient).
- Dynamic appointment booking engine with real-time conflict prevention.
- Redis API caching on high-traffic endpoints (like Admin Dashboard) for performance optimization.
- Asynchronous background jobs using Celery and Redis to handle CSV/PDF generation.

- Mobile-responsive UI utilizing a dark-mode glassmorphic design system.

---

8. Video Presentation

Drive Link:
https://drive.google.com/file/d/19ipkbe2sznxYOkPwx-AF8DBTfIxZAqR5/view?usp=sharing
(Accessible to all with “View” permission.)
